



University of San Carlos | Department of
COMPUTER ENGINEERING

CpE 3104 – Microprocessors

Interfacing Parallel I/O (Numeric Keypad)

UNIT 7: INTERFACING PARALLEL I/O DEVICES

© 2025

Interfacing Numeric Keypad to 8086 Microprocessor

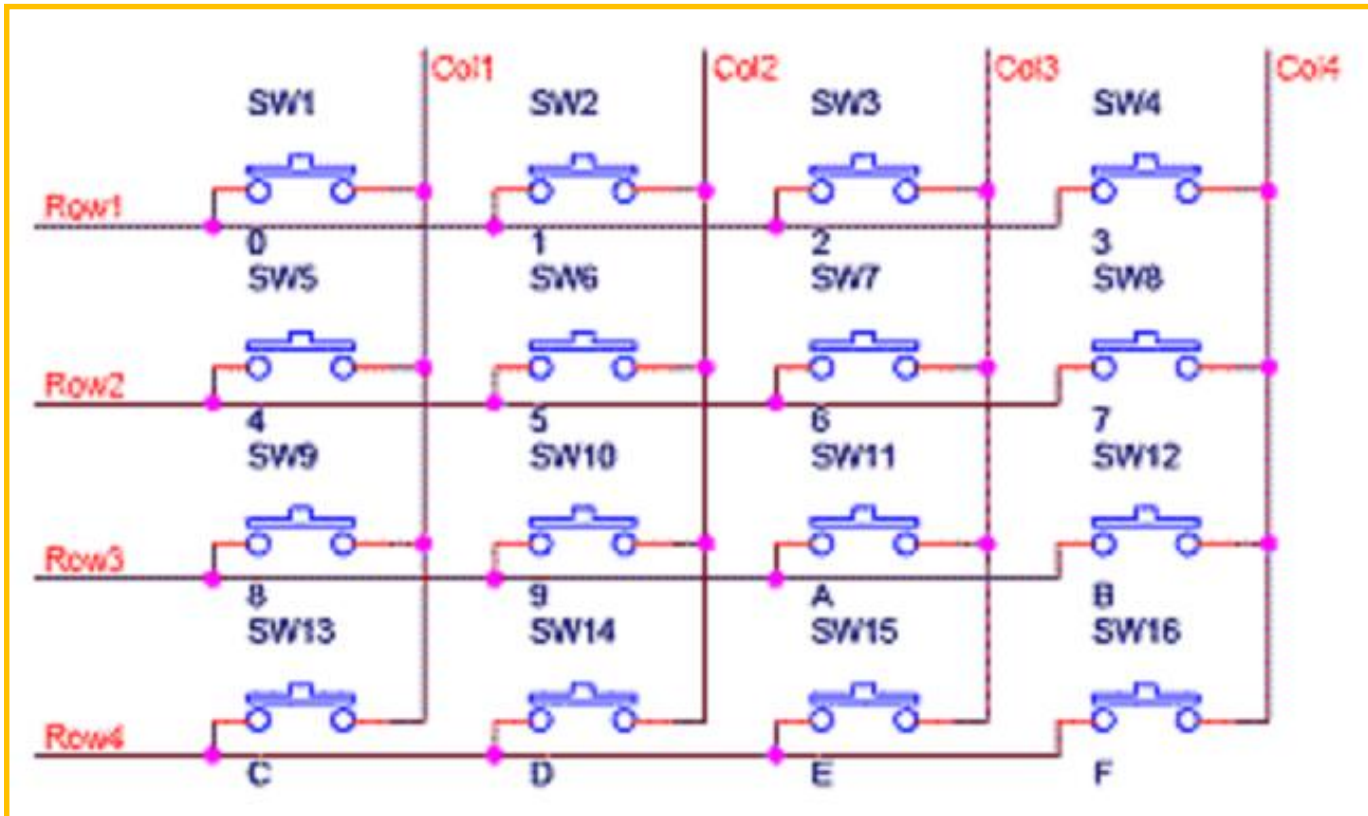
Numeric Keypad

- Most common input device in embedded systems appliances
- Follows the telephony standard layout (3x4 or 4x4)
- Uses array of pushbutton switches with common columns (X) and rows (Y); eliminates individual control of all switches
- Can be interfaced with encoder integrated circuit

Numeric Keypad



Schematic Diagram

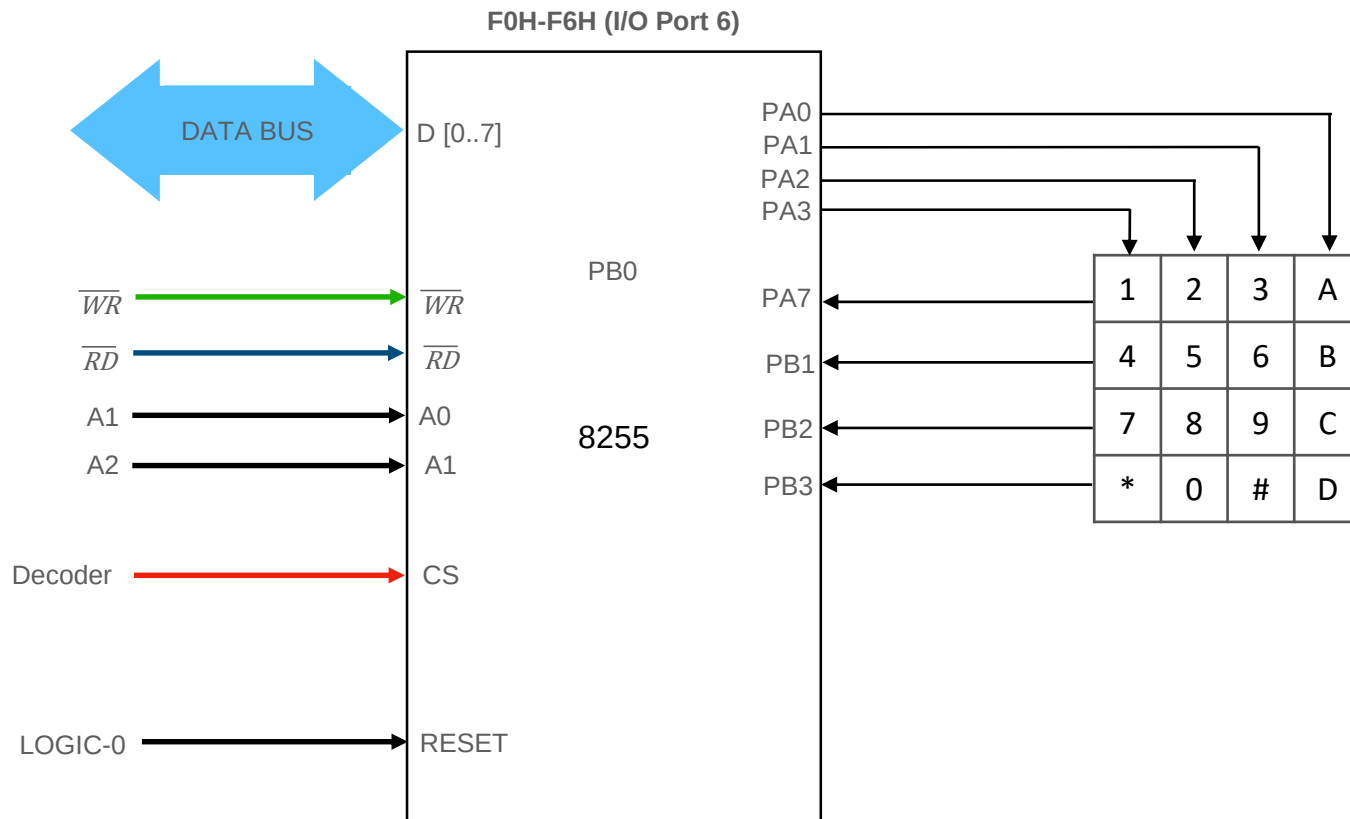


4x4 layout

How to Control?

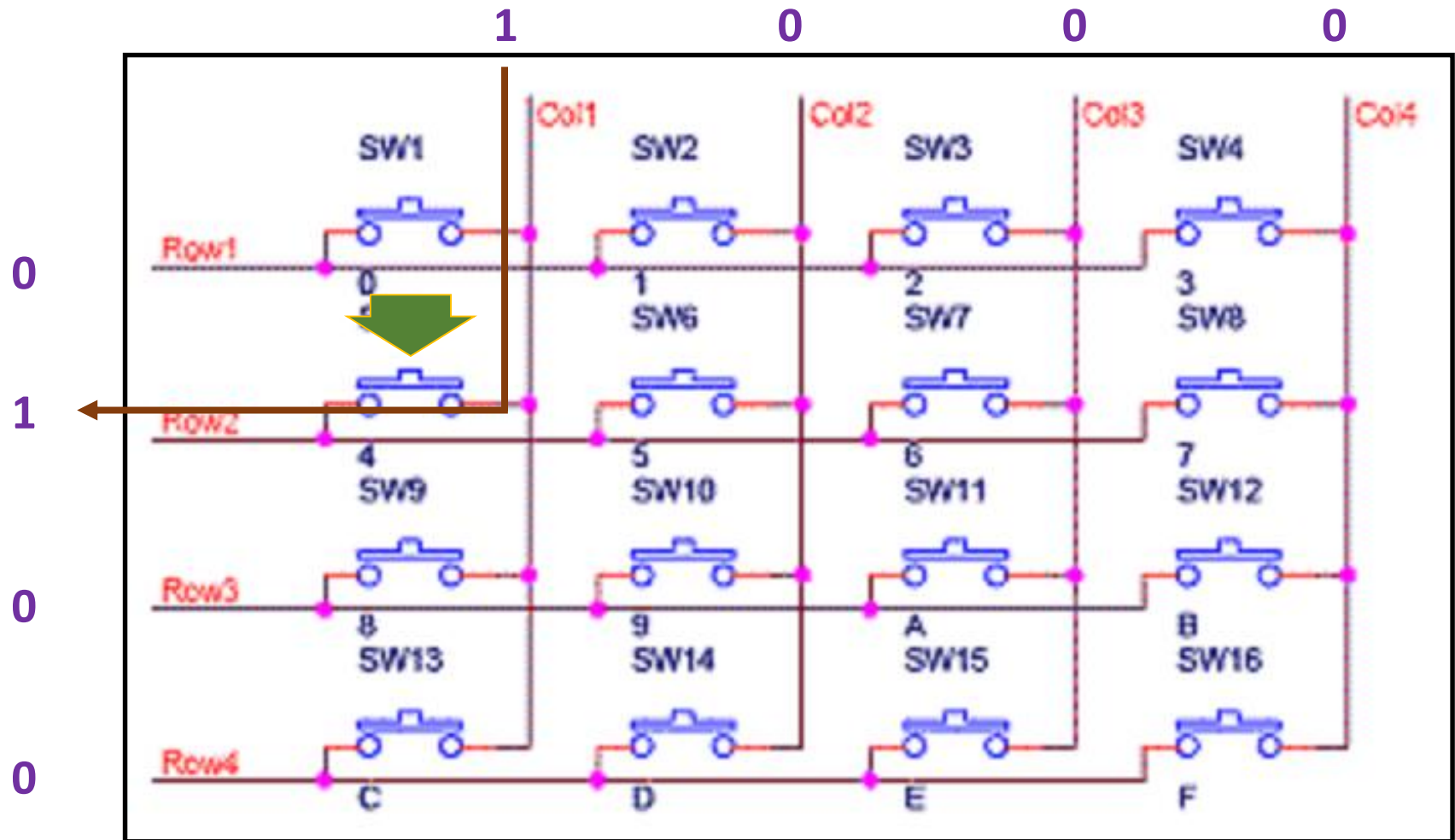
- Supply one of the columns, for example, Column 1 (X1) with logic-1 while the rest is logic-0.
- If the one of the buttons in that column is pressed, current will flow to the corresponding row.
- Therefore, the column and row data will now be used to identify the specific key pressed.
- Move to the next column and read the row data.

Interfacing via 8255 PPI

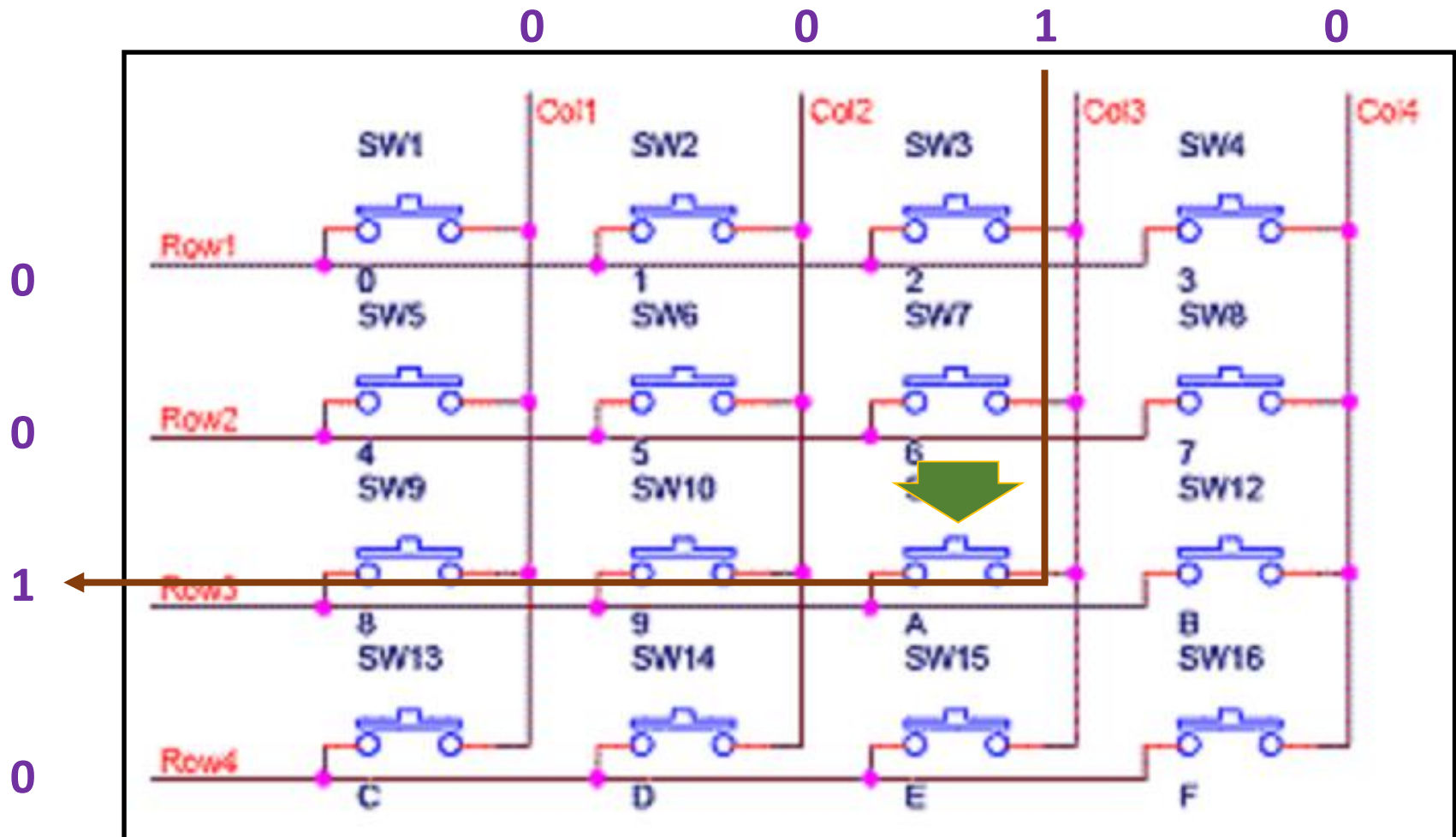


NOTE: PORTA is an output port while PORTB is an input port.

Numeric Keypad



Numeric Keypad

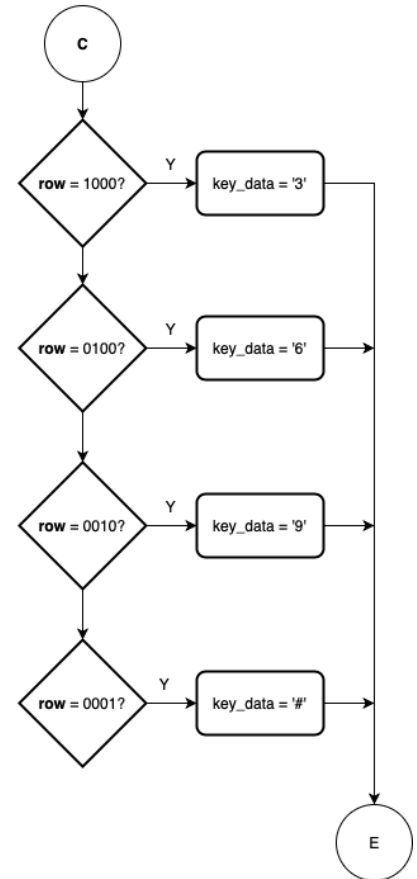
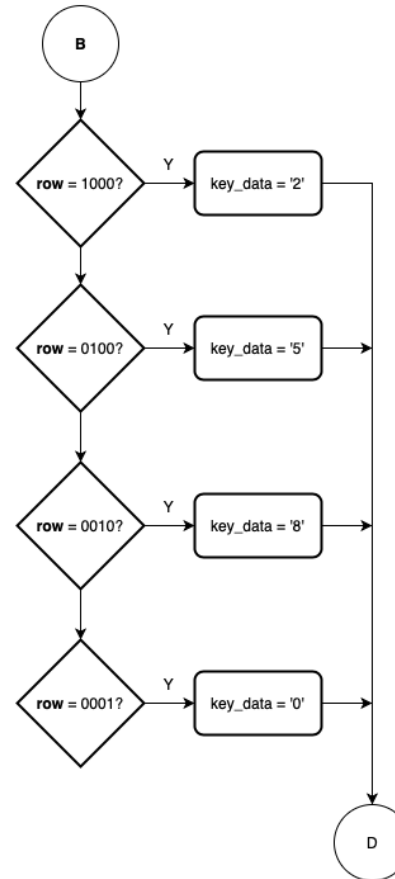
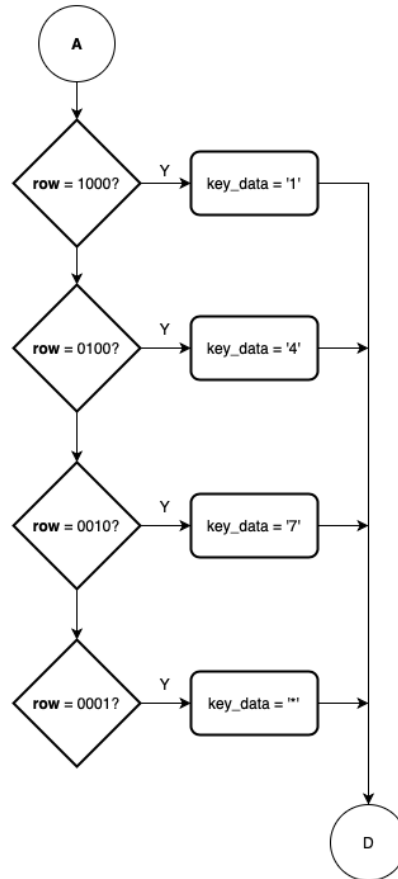
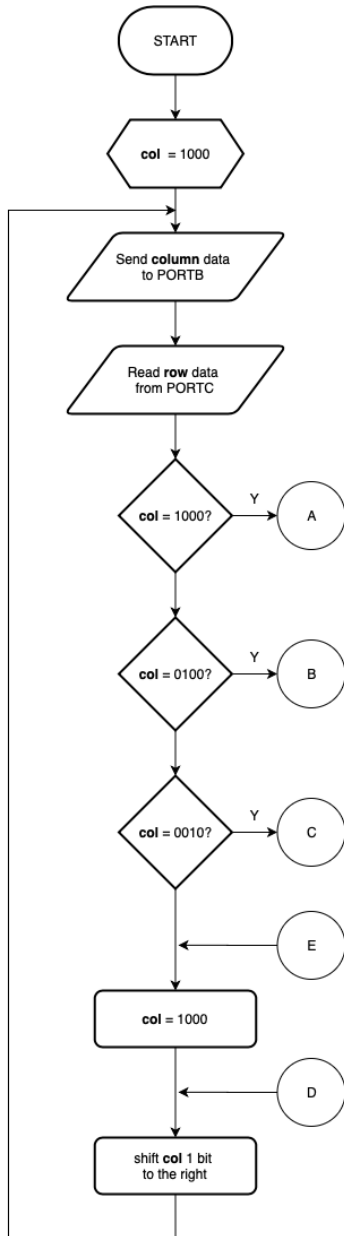




Numeric Keypad

Key	SW No.	Column	Row
1	SW1	1000b	1000b
4	SW5		0100b
7	SW9		0010b
*	SW13		0001b
2	SW2	0100b	1000b
5	SW6		0100b
8	SW10		0010b
0	SW14		0001b
3	SW3	0010b	1000b
6	SW7		0100b
9	SW11		0010b
#	SW15		0001b
A	SW4	0001b	1000b
B	SW8		0100b
C	SW12		0010b
D	SW16		0001b

Flowchart in Controlling a 3x4 Numeric Keypad (4th column is not read)

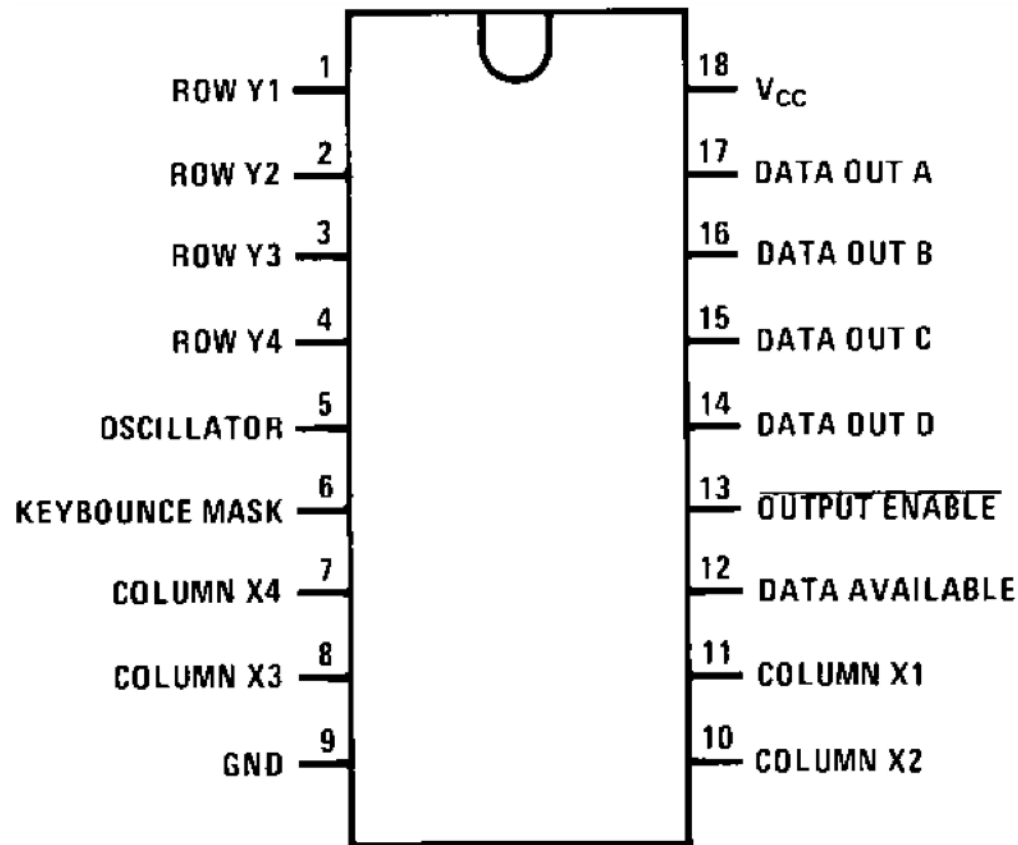


Control Using 74C922

- **74C922*** is a **CMOS key encoder** that encode an array of **SPST** (single-pole, single-throw) switches.
- **Keyboard** (keypad) scanning is implemented automatically by the IC.
- It has an **internal debounce** circuitry.
- **Data Available** (**DAVBL**) output returns a logic-1 when any of the key is pressed.

Pinout of the 74C922 Keypad Encoder

Pin Assignment for DIP

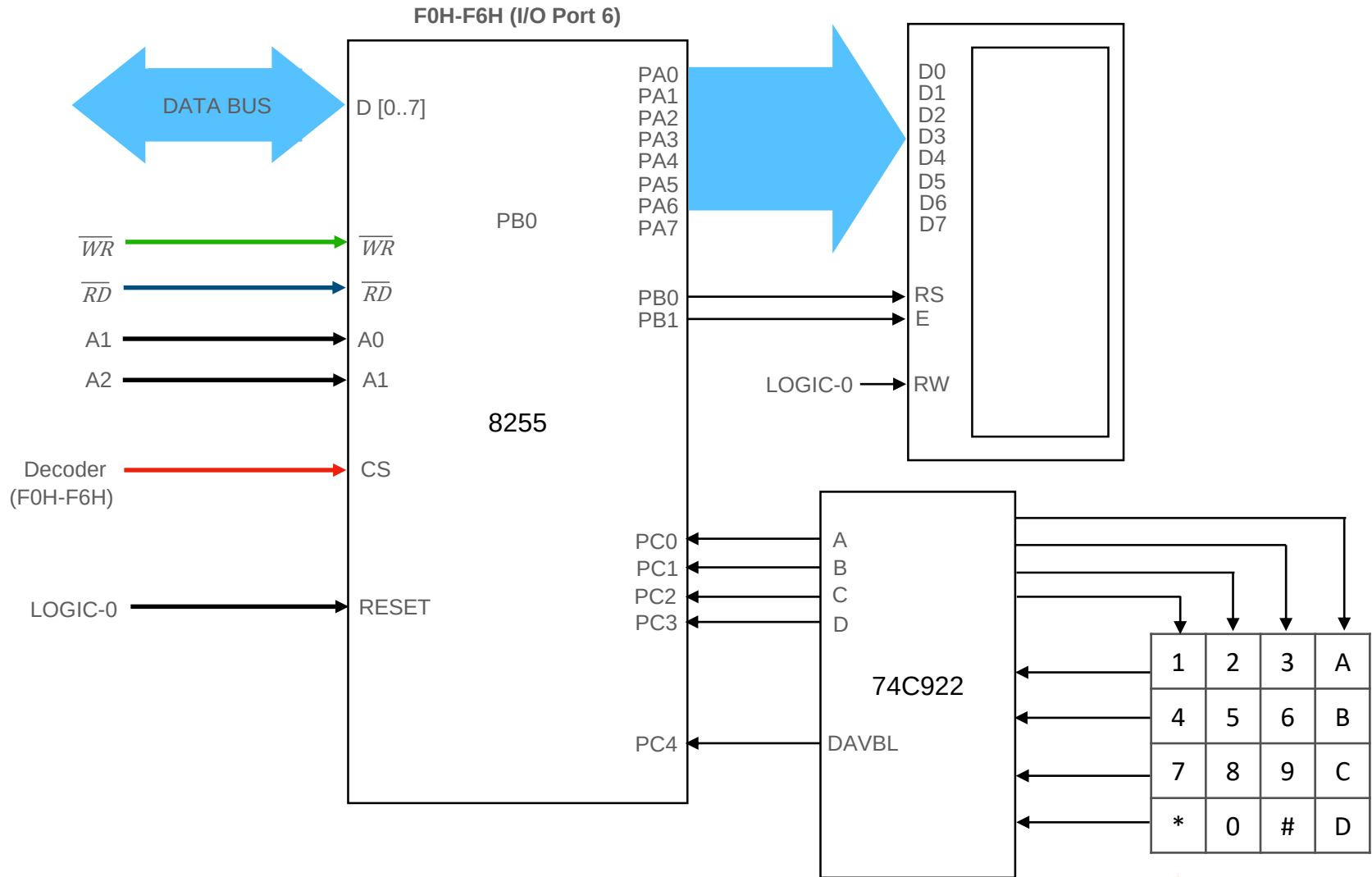


Data from 74C922

Key	74C922 Data Output (bin)	Hex
1	0000b	0x00
2	0001b	0x01
3	0010b	0x02
A	0011b	0x03
4	0100b	0x04
5	0101b	0x05
6	0110b	0x06
B	0111b	0x07
7	1000b	0x08
8	1001b	0x09
9	1010b	0x0A
C	1011b	0x0B
*	1100b	0x0C
0	1101b	0x0D
#	1110b	0x0E
D	1111b	0x0F

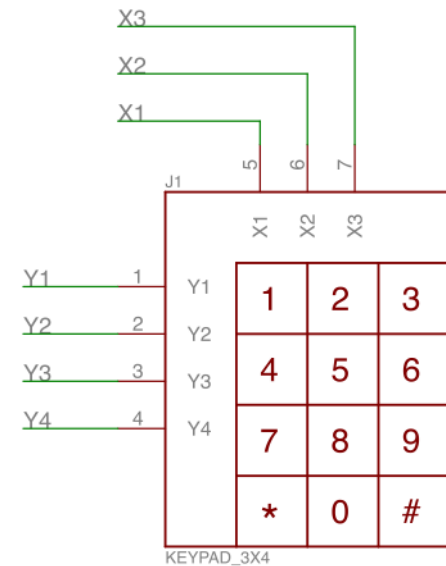
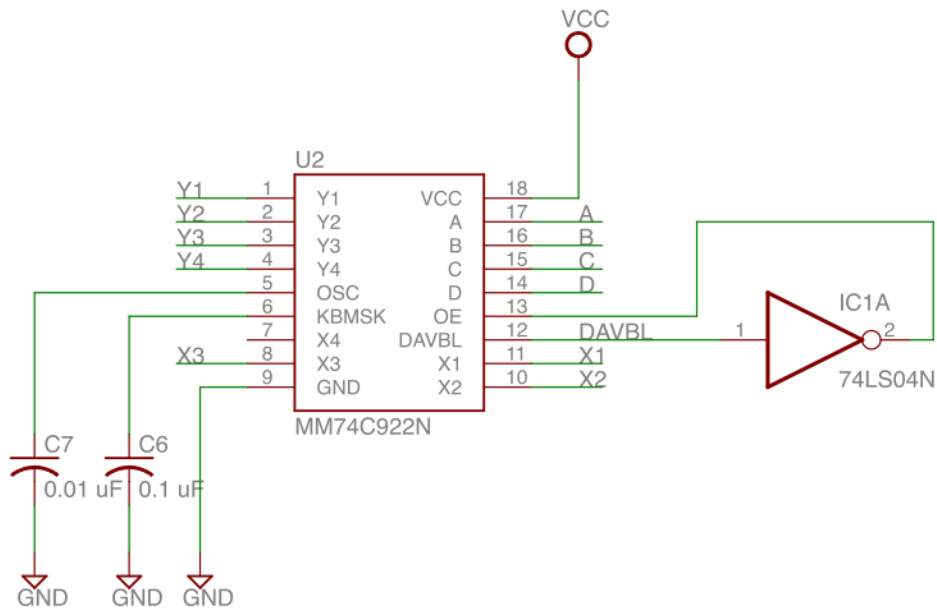
** Keys in yellow cannot be accessed in 3x4 layout*

Interfacing via 8255 PPI



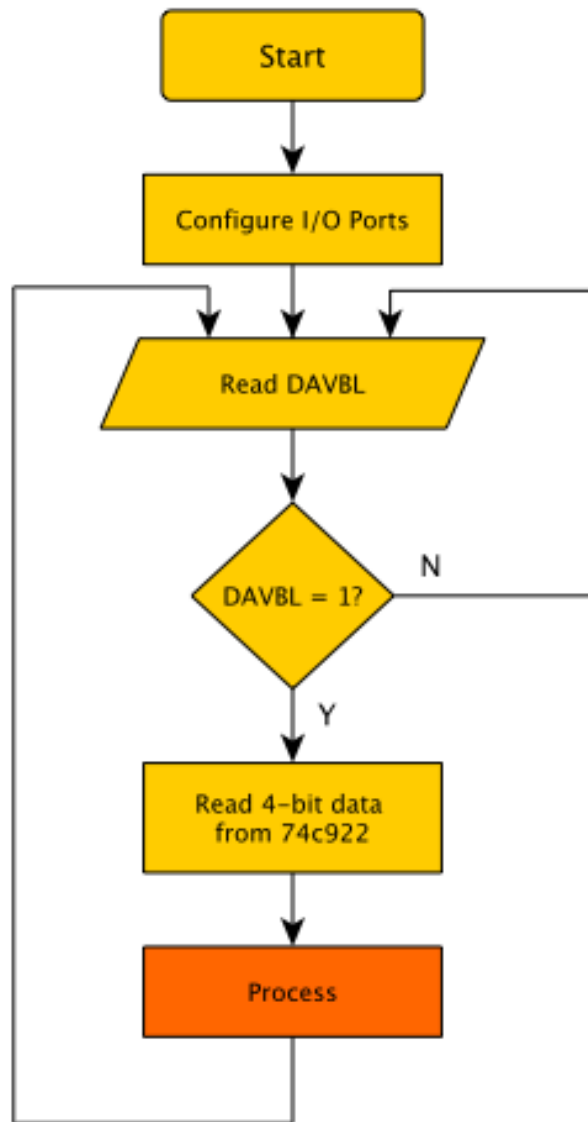
Electrical Connection of 74C922

With 3x4 Keypad



Software Control

- The *DAVBL pin* will go **high** (logic-1) when any key is pressed.
- This can be used as a signal to read the keypad data at A, B, C & D.



Read **DAVBL** before reading
4-bit data from 74C922

Example

- Read keypad data when **DAVBL** is high:

CHECK_DAVBL:

```

MOV DX, PORTC      ; set port of DAVBL(PORTC)
IN AL, DX           ; read PORTC
TEST AL, 10H        ; check if DAVBL is high
JZ CHECK_DAVBL      ; if low then check again
IN AL, DX           ; read 4-bit keypad data
AND AL, 0FH         ; mask upper nibble

```

Software Control Issues

- The example before shows that the keypad data will only be available if **DAVBL is high**. The software will continue to read DAVBL and exit the read loop only when DAVBL be high.
- Not very efficient but it works.
- Most efficient method:
 - external interrupt (will be discussed in Unit 8)
 - scheduling (will be covered in Operating Systems)

Exercise #1

- The **keypad+74C922** and a **single 7-segment display (no decoder)** will be **interfaced to the 8086 via the 8255** at address **E8H-EEH**. 7-segment display is connected to PORTA and 74C922 is connected to PORTC: data (PC0-PC3), DAVBL (PC4).
- The program will display the key pressed on the keypad to the 7-segment display. For keys **“*”** and **“#”**, the display will be **‘-’**.
- Display is **‘0’** initially, then **DAVBL** will be read until it goes **high (logic-1)** then **update the display**.

Exercise #1

```

DATA SEGMENT
    PORTA EQU 0E8H          ; address to access PORTA
    PORTB EQU 0EAH          ; address to access PORTB
    PORTC EQU 0ECH          ; address to access PORTC
    PORT_CON EQU 0EEH       ; address to access Command Register
DATA ENDS

CODE SEGMENT PUBLIC 'CODE'
    ASSUME CS:CODE

START:
    ; Program 8255 (PORTA & PORTB as output, PORTC as input)
    MOV DX, PORT_CON        ; set port to Command Register
    MOV AL, 89H             ; set command byte

    OUT DX, AL              ; send data in AL to Command Register

CHECK_DAVBL:
    MOV DX, PORTC           ; set port of DAVBL(PORTC)
    IN AL, DX               ; read PORTC
    TEST AL, 10H            ; check if DAVBL is high
    JZ CHECK_DAVBL         ; if low then check again
    IN AL, DX               ; read 4-bit keypad data

    AND AL, 0FH             ; mask upper nibble
    CMP AL, 00H             ; check if key pressed is 1 (00H)
    JE D1                   ; display 1
    CMP AL, 01H             ; check if key pressed is 2 (01H)
    JE D2                   ; display 2
    CMP AL, 02H             ; check if key pressed is 3 (02H)
    JE D3                   ; display 3
    CMP AL, 04H             ; check if key pressed is 4 (04H)
    JE D4                   ; display 4
    CMP AL, 05H             ; check if key pressed is 5 (05H)
    JE D5                   ; display 5
    CMP AL, 06H             ; check if key pressed is 6 (06H)
    JE D6                   ; display 6
    CMP AL, 08H             ; check if key pressed is 7 (08H)
    JE D7                   ; display 7

```

Exercise #1

```

        CMP AL, 09H          ; check if key pressed is 8 (09H)
        JE D8                ; display 8
        CMP AL, 0AH          ; check if key pressed is 9 (0AH)
        JE D9                ; display 9
        CMP AL, 0CH          ; check if key pressed is * (0CH)
        JE D10               ; display -
        CMP AL, 0DH          ; check if key pressed is 0 (0DH)
        JE D0                ; display 0
        CMP AL, 0EH          ; check if key pressed is * (0EH)
        JE D10               ; display -
        CALL DELAY_1MS

        JMP CHECK_DAVBL
D0:     MOV AL, 00111111B    ; display '0'
        JMP CONT
D1:     MOV AL, 00000110B    ; display '1'
        JMP CONT
D2:     MOV AL, 01011011B    ; display '2'
        JMP CONT
D3:     MOV AL, 01001111B    ; display '3'
        JMP CONT
D4:     MOV AL, 01100110B.    ; display '4'
        JMP CONT
D5:     MOV AL, 01101101B    ; display '5'
        JMP CONT
D6:     MOV AL, 01111101B    ; display '6'
        JMP CONT
D7:     MOV AL, 00000111B    ; display '7'
        JMP CONT
D8:     MOV AL, 01111111B    ; display '8'
        JMP CONT
D9:     MOV AL, 01100111B.    ; display '9'
        JMP CONT
D10:    MOV AL, 01000000B    ; display '-'
```

Exercise #1

```
CONT:
    MOV DX, PORTA
    OUT DX, AL
    CALL DELAY_1MS
    JMP CHECK_DAVBL
DELAY_1MS:
    MOV BX, 02CAH
L1:
    NOP
    JNZ
    L1

    RET
CODE ENDS
END START
```

Exercise #2

- The keypad+74C922 and LCD will be interfaced to the 8086 via the 8255 at address F0H-F6H.
- LCD configuration: 8-bit, cursor off, blink off, increment cursor, display shift off
- LCD is initialized first, then check keypad input.
 - if **1** is pressed, print “University of” middle of 1st line
 - if **2** is pressed, print “San Carlos” middle of 2nd line
 - if **3** is pressed, print “Department of” middle of 3rd line
 - if **4** is pressed, print “Computer Engineering” middle of 4th line
 - if ***** is pressed, clear the display
 - if any other key is pressed, do nothing

Exercise #2

```

DATA SEGMENT
    PORTA EQU 0F0H          ; address to access PORTA
    PORTB EQU 0F2H          ; address to access PORTB
    PORTC EQU 0F4H          ; address to access PORTC
    PORT_CON EQU 0F6H       ; address to access Command Register
DATA ENDS

CODE SEGMENT PUBLIC 'CODE'
    ASSUME CS:CODE

START:
    ; Program 8255 (PORTA & PORTB as output, PORTC as input)
    MOV DX, PORT_CON        ; set port to Command Register
    MOV AL, 89H             ; set command byte

    OUT DX, AL              ; send data in AL to Command Register

    CALL INIT_LCD           ; initialize LCD

CHECK_DAVBL:
    MOV DX, PORTC           ; set port of DAVBL(PORTC)
    IN AL, DX               ; read PORTC
    TEST AL, 10H            ; check if DAVBL is high
    JZ CHECK_DAVBL         ; if low then check again
    IN AL, DX               ; read 4-bit keypad data

    AND AL, 0FH             ; mask upper nibble
    CMP AL, 00H            ; check if key pressed is 1 (00H)
    JE PRNT_LINE1          ; print line 1
    CMP AL, 01H            ; check if key pressed is 2 (01H)
    JE PRNT_LINE2          ; print line 2
    CMP AL, 02H            ; check if key pressed is 3 (02H)
    JE PRNT_LINE3          ; print line 3
    CMP AL, 04H            ; check if key pressed is 4 (04H)
    JE PRNT_LINE4          ; print line 4
    CMP AL, 0CH            ; check if key pressed is * (0CH)

    JE CLR_DISP
    JMP CHECK_DAVBL

```

Exercise #2

```

; display string "University of" at middle of 1st line
PRNT_LINE1:
    MOV AL, 83H      ; position cursor at 1st line, 4th column
    CALL INST_CTRL ; send command to LCD
    MOV AL, 'U'      ; set ASCII code of character 'U'
    CALL DATA_CTRL ; display the character
    MOV AL, 'n'      ; set ASCII code of character 'n'
    CALL DATA_CTRL ; display the character
    MOV AL, 'i'      ; set ASCII code of character 'i'
    CALL DATA_CTRL ; display the character
    MOV AL, 'v'      ; set ASCII code of character 'v'
    CALL DATA_CTRL ; display the character
    MOV AL, 'e'      ; set ASCII code of character 'e'
    CALL DATA_CTRL ; display the character
    MOV AL, 'r'      ; set ASCII code of character 'r'
    CALL DATA_CTRL ; display the character
    MOV AL, 's'      ; set ASCII code of character 's'
    CALL DATA_CTRL ; display the character
    MOV AL, 'i'      ; set ASCII code of character 'i'
    CALL DATA_CTRL ; display the character
    MOV AL, 't'      ; set ASCII code of character 't'
    CALL DATA_CTRL ; display the character
    MOV AL, 'y'      ; set ASCII code of character 'y'
    CALL DATA_CTRL ; display the character
    MOV AL, ' '      ; set ASCII code of character ' '
    CALL DATA_CTRL ; display the character
    MOV AL, 'o'      ; set ASCII code of character 'o'
    CALL DATA_CTRL ; display the character
    MOV AL, 'f'      ; set ASCII code of character 'f'
    CALL DATA_CTRL ; display the character
    JMP CHECK_DAVBL

```

Exercise #2

```

        ; display string "San Carlos" at middle of 2nd line
PRNT_LINE2:
    MOV AL, 0C5H      ; position cursor at 2nd line, 6th column
    CALL INST_CTRL    ; send command to LCD
    MOV AL, 'S'        ; set ASCII code of character 'S'
    CALL DATA_CTRL    ; display the character
    MOV AL, 'a'        ; set ASCII code of character 'a'
    CALL DATA_CTRL    ; display the character
    MOV AL, 'n'        ; set ASCII code of character 'n'
    CALL DATA_CTRL    ; display the character
    MOV AL, ' '        ; set ASCII code of character ' '
    CALL DATA_CTRL    ; display the character
    MOV AL, 'C'        ; set ASCII code of character 'C'
    CALL DATA_CTRL    ; display the character
    MOV AL, 'a'        ; set ASCII code of character 'a'
    CALL DATA_CTRL    ; display the character
    MOV AL, 'r'        ; set ASCII code of character 'r'
    CALL DATA_CTRL    ; display the character
    MOV AL, 'l'        ; set ASCII code of character 'l'
    CALL DATA_CTRL    ; display the character
    MOV AL, 'o'        ; set ASCII code of character 'o'
    CALL DATA_CTRL    ; display the character
    MOV AL, 's'        ; set ASCII code of character 's'
    CALL DATA_CTRL    ; display the character
    JMP CHECK_DAVBL

```

Exercise #2

```

; display string "Department of" at middle of 3rd line
PRNT_LINE3:
    MOV AL, 97H      ; position cursor at 3rd line, 4th column
    CALL INST_CTRL ; send command to LCD
    MOV AL, 'D'      ; set ASCII code of character 'D'
    CALL DATA_CTRL ; display the character
    MOV AL, 'e'      ; set ASCII code of character 'e'
    CALL DATA_CTRL ; display the character
    MOV AL, 'p'      ; set ASCII code of character 'p'
    CALL DATA_CTRL ; display the character
    MOV AL, 'a'      ; set ASCII code of character 'a'
    CALL DATA_CTRL ; display the character
    MOV AL, 'r'      ; set ASCII code of character 'r'
    CALL DATA_CTRL ; display the character
    MOV AL, 't'      ; set ASCII code of character 't'
    CALL DATA_CTRL ; display the character
    MOV AL, 'm'      ; set ASCII code of character 'm'
    CALL DATA_CTRL ; display the character
    MOV AL, 'e'      ; set ASCII code of character 'e'
    CALL DATA_CTRL ; display the character
    MOV AL, 'n'      ; set ASCII code of character 'n'
    CALL DATA_CTRL ; display the character
    MOV AL, 't'      ; set ASCII code of character 't'
    CALL DATA_CTRL ; display the character
    MOV AL, ' '      ; set ASCII code of character ' '
    CALL DATA_CTRL ; display the character
    MOV AL, 'o'      ; set ASCII code of character 'o'
    CALL DATA_CTRL ; display the character
    MOV AL, 'f'      ; set ASCII code of character 'f'
    CALL DATA_CTRL ; display the character
    JMP CHECK_DAVBL

```

Exercise #2

```

; display string "Computer Engineering" at middle of 4th line
PRNT_LINE4:
    MOV AL, 0D4H      ; position cursor at 4th line, 1st column
    CALL INST_CTRL    ; send command to LCD
    MOV AL, 'C'       ; set ASCII code of character 'C'
    CALL DATA_CTRL   ; display the character
    MOV AL, 'o'       ; set ASCII code of character 'o'
    CALL DATA_CTRL   ; display the character
    MOV AL, 'm'       ; set ASCII code of character 'm'
    CALL DATA_CTRL   ; display the character
    MOV AL, 'p'       ; set ASCII code of character 'p'
    CALL DATA_CTRL   ; display the character
    MOV AL, 'u'       ; set ASCII code of character 'u'
    CALL DATA_CTRL   ; display the character
    MOV AL, 't'       ; set ASCII code of character 't'
    CALL DATA_CTRL   ; display the character
    MOV AL, 'e'       ; set ASCII code of character 'e'
    CALL DATA_CTRL   ; display the character
    MOV AL, 'r'       ; set ASCII code of character 'r'
    CALL DATA_CTRL   ; display the character
    MOV AL, ' '       ; set ASCII code of character ' '
    CALL DATA_CTRL   ; display the character
    MOV AL, 'E'       ; set ASCII code of character 'E'
    CALL DATA_CTRL   ; display the character
    MOV AL, 'n'       ; set ASCII code of character 'n'
    CALL DATA_CTRL   ; display the character
    MOV AL, 'g'       ; set ASCII code of character 'g'
    CALL DATA_CTRL   ; display the character
    MOV AL, 'i'       ; set ASCII code of character 'i'
    CALL DATA_CTRL   ; display the character
    MOV AL, 'n'       ; set ASCII code of character 'n'

```

Exercise #2

```

CALL DATA_CTRL ; display the character
MOV AL, 'e'      ; set ASCII code of character 'e'
CALL DATA_CTRL ; display the character
MOV AL, 'e'      ; set ASCII code of character 'e'
CALL DATA_CTRL ; display the character
MOV AL, 'r'      ; set ASCII code of character 'r'
CALL DATA_CTRL ; display the character
MOV AL, 'i'      ; set ASCII code of character 'i'
CALL DATA_CTRL ; display the character
MOV AL, 'n'      ; set ASCII code of character 'n'
CALL DATA_CTRL ; display the character
MOV AL, 'g'      ; set ASCII code of character 'g'
CALL DATA_CTRL ; display the character

JMP CHECK_DAVBL
; clear the display
CLR_DISP:
MOV AL, 01H      ; set clear display instruction (01H)
CALL INST_CTRL ; send instruction to LCD

JMP CHECK_DAVBL
; this function will send an 8-bit instruction to the LCD
INST_CTRL:
PUSH AX          ; preserve value of AL
MOV DX, PORTA    ; set port of LCD data bus (PORTA)
OUT DX, AL       ; write data in AL to PORTA
MOV DX, PORTB    ; set port of LCD control lines (PORTB)
MOV AL, 02H      ; E=1, RS=0 (access instruction reg)
OUT DX, AL       ; write data in AL to PORTB
CALL DELAY_1MS   ; delay for 1 ms
MOV DX, PORTB    ; set port of LCD control lines (PORTB)
MOV AL, 00H      ; E=0, RS=0
OUT DX, AL       ; write data in AL to PORTB
POP AX          ; restore value of AL

RET

```

Exercise #2

```

        ; this function will send an 8-bit data (character) to the LCD
DATA_CTRL:
    PUSH AX          ; preserve value of AL
    MOV DX, PORTA    ; set port of LCD data bus (PORTA)
    OUT DX, AL       ; write data in AL to PORTA
    MOV DX, PORTB    ; set port of LCD control lines (PORTB)
    MOV AL, 03H      ; E=1, RS=1 (access data reg)
    OUT DX, AL       ; write data in AL to PORTB
    CALL DELAY_1MS   ; delay for 1 ms
    MOV DX, PORTB    ; set port of LCD control lines (PORTB)
    MOV AL, 01H      ; E=0, RS=1
    OUT DX, AL       ; write data in AL to PORTB
    POP AX           ; restore value of AL

    RET
    ; this function will initialize the LCD
INIT_LCD:
    MOV AL, 38H      ; 8-bit interface, dual-line display
    CALL INST_CTRL   ; write instruction to LCD
    MOV AL, 08H      ; display off, cursor off, blink off
    CALL INST_CTRL   ; write instruction to LCD
    MOV AL, 01H      ; clear display
    CALL INST_CTRL   ; write instruction to LCD
    MOV AL, 06H      ; increment cursor, display shift off
    CALL INST_CTRL   ; write instruction to LCD
    MOV AL, 0CH      ; display on, cursor off, blink off
    CALL INST_CTRL   ; write instruction to LCD

    RET
DELAY_1MS:
    MOV BX, 02CAH
L1:
    NOP
    JNZ
    L1

    RET
CODE ENDS
END START

```



University of San Carlos | Department of
COMPUTER ENGINEERING

CpE 3104 – Microprocessors

End of Lecture

UNIT 7: INTERFACING PARALLEL I/O DEVICES

© 2025